Machine Learning • Bab 6

# 6.16 dan 6.17 Implementasi Python KNN & Naive Bayes

Modul lengkap: Praktikum 6.1 – 6.7 dan Latihan Praktikum 6.17, dengan penjelasan tiap langkah, output asli, dan interpretasi

Disusun oleh

**Ahmad Dandi Subhani**

202343500126

Buka Notebook di Google Colab

## 6.16. Implementasi Python

Bagian ini mempraktikkan dua algoritma klasifikasi klasik: K-Nearest Neighbors (KNN) dan Naive Bayes, beserta penerapannya pada klasifikasi teks. Dataset yang dipakai adalah Wine dari sklearn.datasets (178 sampel, 13 fitur kimia, 3 kelas dengan distribusi 59, 71, dan 48).

### Praktikum 6.1 Persiapan Library

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
```

**Penjelasan:** numpy untuk komputasi dan matplotlib untuk visualisasi. Dari sklearn diimpor load_wine (dataset), train_test_split (split data), StandardScaler (scaling untuk KNN), KNeighborsClassifier dan GaussianNB (dua model yang dibandingkan), serta accuracy_score untuk evaluasi.

### Praktikum 6.2 Import Dataset

```
data = load_wine(as_frame=True)
X, y = data.data, data.target
print("Ukuran data:", X.shape)
print("Distribusi kelas:", np.bincount(y))
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42, stratify=y)
```

```
Ukuran data: (178, 13)
Distribusi kelas: [59 71 48]
```

**Penjelasan:** Dataset Wine dimuat sebagai DataFrame agar nama fiturnya terbaca. Output menunjukkan 178 sampel dengan 13 fitur kimia dan tiga kelas (59, 71, 48). Data dibagi 80 persen latih (142 sampel) dan 20 persen uji (36 sampel); stratify=y menjaga proporsi kelas tetap sama, random_state=42 membuat hasilnya dapat diulang.

### Praktikum 6.3 Preprocessing (Scaling untuk KNN)

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
# Catatan: Naive Bayes tidak memerlukan scaling,
# namun KNN sangat sensitif terhadap skala fitur.
```

**Penjelasan:** KNN bekerja dengan menghitung jarak antar titik, sehingga sangat sensitif terhadap skala fitur: fitur dengan rentang besar akan mendominasi jarak. Karena itu data di-scaling dengan StandardScaler; fit hanya dilakukan di data latih lalu hasilnya dipakai untuk mentransform data uji, agar tidak ada kebocoran informasi. Naive Bayes tidak butuh scaling karena ia bekerja dengan distribusi probabilitas tiap fitur, bukan jarak.

## Praktikum 6.4 Training Model

```
knn = KNeighborsClassifier(n_neighbors=5,
    weights="distance")
knn.fit(X_train_scaled, y_train)

nb = GaussianNB()
nb.fit(X_train, y_train)
```

```
GaussianNB()
```

**Penjelasan:** Dua model dilatih. **KNN** dengan n_neighbors=5 dan weights="distance" (tetangga yang lebih dekat diberi bobot lebih besar), dilatih pada data yang sudah di-scaling. **Gaussian Naive Bayes** dilatih pada data asli tanpa scaling. Output GaussianNB() adalah representasi objek model yang berhasil dibuat, artinya kedua model sudah siap dipakai.

## Praktikum 6.5 Evaluasi

```
acc_knn = accuracy_score(y_test, knn.predict(X_test_scaled))
acc_nb = accuracy_score(y_test, nb.predict(X_test))
print(f"Akurasi KNN (k=5, weighted): {acc_knn:.3f}")
print(f"Akurasi Gaussian Naive Bayes: {acc_nb:.3f}")
```

```
Akurasi KNN (k=5, weighted): 0.972
Akurasi Gaussian Naive Bayes: 0.972
```

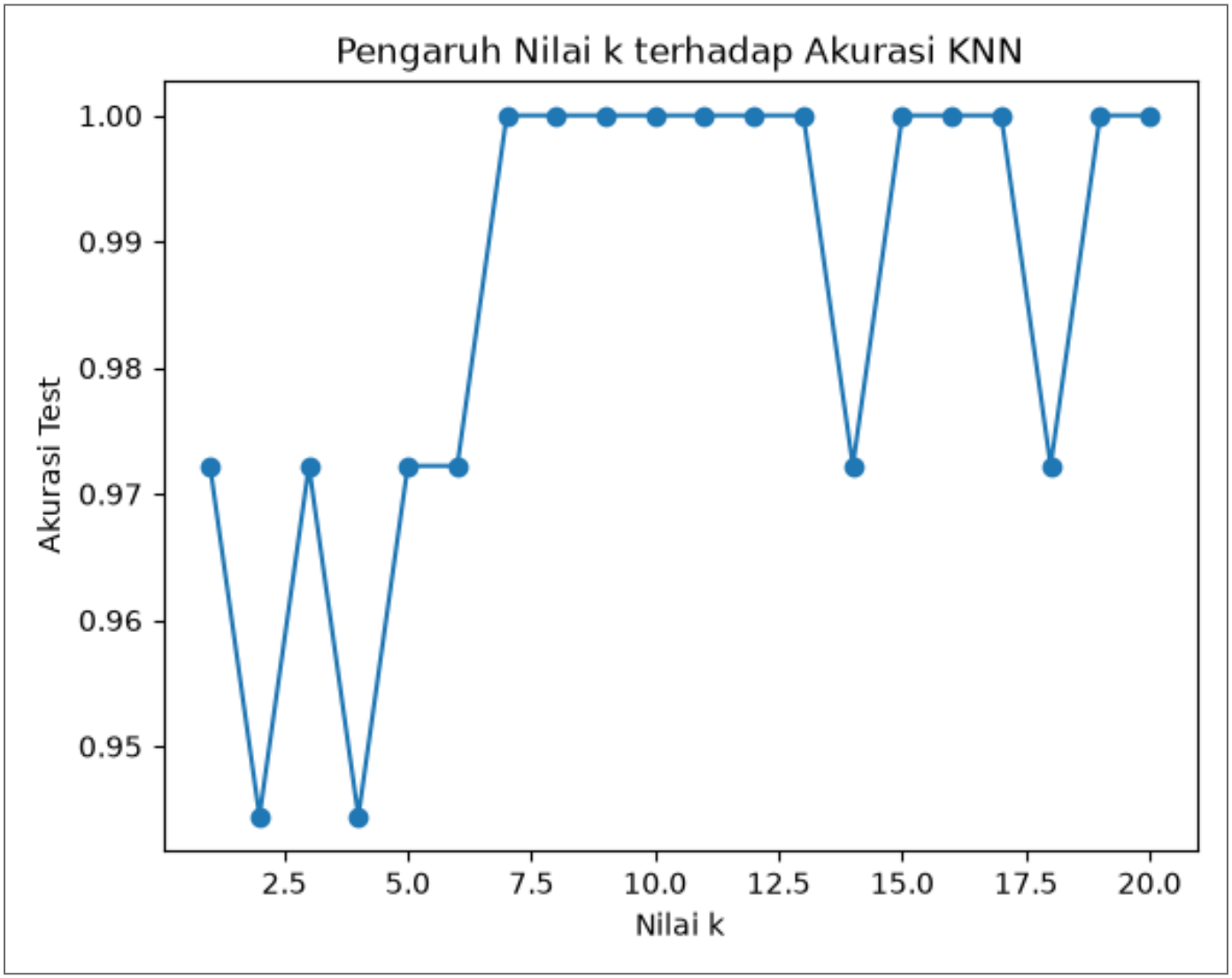
**Penjelasan:** Kedua model diuji pada 36 data uji dan hasilnya **seri**: akurasi 0,972, artinya masing-masing benar 35 dari 36 data dan hanya 1 yang salah. Dua algoritma dengan cara kerja yang sangat berbeda sampai pada kesimpulan yang sama, artinya pola kelas pada data Wine memang cukup jelas dan terpisah dengan baik.

| Model | Akurasi test |
|---|---|
| KNN (k=5, weighted) | 0,972 |
| Gaussian Naive Bayes | 0,972 |

## Praktikum 6.6 Visualisasi (Pengaruh Nilai k)

```
akurasi_k = []
nilai_k = range(1, 21)
for k in nilai_k:
    model = KNeighborsClassifier(n_neighbors=k).fit(X_train_scaled, y_train)
    akurasi_k.append(accuracy_score(y_test, model.predict(X_test_scaled)))

plt.plot(nilai_k, akurasi_k, marker='o')
plt.xlabel("Nilai k")
plt.ylabel("Akurasi Test")
plt.title("Pengaruh Nilai k terhadap Akurasi KNN")
plt.show()
```




Gambar 6.1: Pengaruh nilai k terhadap akurasi KNN pada data Wine.

**Penjelasan:** Nilai k dicoba dari 1 sampai 20 dan akurasinya diplot. Pada k kecil (1 sampai 6) akurasi berfluktuasi antara 0,944 dan 0,972 karena model hanya melihat sedikit tetangga sehingga sensitif terhadap noise. Mulai **k=7** akurasi mencapai **1,000** dan relatif stabil, jadi k=7 adalah pilihan optimal pertama pada eksperimen ini. Pelajarannya: k terlalu kecil

membuat model tidak stabil, sedangkan k terlalu besar membuat batas keputusan terlalu halus.

### Praktikum 6.7 Klasifikasi Teks dengan Multinomial Naive Bayes

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB

teks = ["dapatkan diskon gratis sekarang",
        "rapat tim jam 10 pagi",
        "menangkan hadiah gratis klik disini",
        "laporan keuangan bulan ini", "diskon spesial hari ini",
        "jadwal kuliah minggu depan"]
label = [1, 0, 1, 0, 1, 0]  # 1 = spam, 0 = bukan spam

vectorizer = CountVectorizer()
X_teks = vectorizer.fit_transform(teks)
model = MultinomialNB(alpha=1.0)  # alpha = parameter Laplace Smoothing
model.fit(X_teks, label)

teks_baru = ["diskon gratis hari ini"]
prediksi = model.predict(vectorizer.transform(teks_baru))
print("Prediksi:",
      "Spam" if prediksi[0]==1 else "Bukan Spam")
```

```
Prediksi: Spam
```

**Penjelasan:** Naive Bayes juga jago di data teks, dan di sini dipakai MultinomialNB yang cocok untuk fitur hitungan kata. Enam kalimat Bahasa Indonesia diubah menjadi vektor hitungan kata dengan CountVectorizer, lalu model dilatih membedakan spam (1) dan bukan spam (0). Parameter alpha=1.0 adalah Laplace smoothing agar kata yang belum pernah muncul tidak membuat probabilitas nol. Kalimat baru "diskon gratis hari ini" diprediksi **Spam** dengan benar, karena kata "diskon" dan "gratis" dominan muncul di kalimat spam pada data latih.

## 6.16.1. Interpretasi

Hasilnya menarik: KNN dan Gaussian Naive Bayes **seri** di akurasi 0,972 (35 dari 36 data uji benar, hanya 1 yang salah). Dua algoritma dengan cara kerja yang sangat berbeda sampai pada kesimpulan yang sama di dataset ini.

**Temuan dari hasil praktik**

- **Seri 0,972 bukan kebetulan:** kedua model benar 35 dari 36 data uji. Pola kelas pada data Wine memang cukup jelas dan terpisah dengan baik.
- **Scaling itu wajib buat KNN.** Tanpa scaling, fitur dengan rentang besar akan mendominasi perhitungan jarak dan akurasi anjlok. Naive Bayes tidak peduli scaling karena ia membandingkan distribusi probabilitas, bukan jarak.
- **Nilai k menentukan karakter model.** Pada k kecil (1 sampai 6) akurasi berfluktuasi antara 0,944 dan 0,972; mulai k=7 akurasi mencapai 1,000 dan stabil. Jadi k=7 adalah pilihan optimal pertama di eksperimen ini.
- **Tidak ada algoritma yang selalu menang.** Di dataset Wine keduanya seri 0,972, tetapi di dataset Breast Cancer (Latihan 1) KNN unggul 0,9561 berbanding 0,9386. Algoritma terbaik tergantung karakter data.

Kesimpulannya, KNN dan Naive Bayes sama-sama layak untuk klasifikasi tabular berukuran sedang seperti Wine. KNN unggul saat pola lokal data kuat dan scaling diperhatikan; Naive Bayes unggul saat kecepatan prediksi dan kesederhanaan asumsi menjadi prioritas. Untuk teks, MultinomialNB terbukti praktis dan akurat.

## 6.17. Latihan Praktikum

Bagian ini berisi dua latihan mandiri dari modul Bab 6. Latihan 1 membandingkan KNN dan Naive Bayes dari sisi akurasi dan kecepatan pada dataset Breast Cancer, sedangkan Latihan 2 menerapkan Multinomial Naive Bayes pada data teks Bahasa Indonesia.

### Latihan 1 KNN vs Naive Bayes (Akurasi, Waktu Pelatihan, Waktu Prediksi)

---

**Soal:** Gunakan dataset load_breast_cancer dari Scikit-learn. Latih model KNN (dengan scaling) dan Gaussian Naive Bayes. Bandingkan akurasi, waktu pelatihan, dan waktu prediksi keduanya menggunakan modul time Python. Kunci latihannya: KNN dilatih pada data yang sudah di-scaling, Naive Bayes pada data asli. Waktu diukur dengan time.perf_counter() tepat sebelum dan sesudah fit() (waktu pelatihan) serta predict() (waktu prediksi).

```
import time
from sklearn.datasets import load_breast_cancer

data_bc = load_breast_cancer(as_frame=True)
Xb, yb = data_bc.data, data_bc.target
Xb_train, Xb_test, yb_train, yb_test = train_test_split(
    Xb, yb, test_size=0.2, random_state=42, stratify=yb)

scaler_bc = StandardScaler()
Xb_train_scaled = scaler_bc.fit_transform(Xb_train)
Xb_test_scaled = scaler_bc.transform(Xb_test)

# KNN: catat waktu pelatihan dan waktu prediksi
t0 = time.perf_counter()
knn_bc = KNeighborsClassifier(n_neighbors=5, weights="distance")
knn_bc.fit(Xb_train_scaled, yb_train)
waktu_train_knn = time.perf_counter() - t0

t0 = time.perf_counter()
pred_knn = knn_bc.predict(Xb_test_scaled)
waktu_pred_knn = time.perf_counter() - t0

# Gaussian Naive Bayes: catat waktu pelatihan dan waktu prediksi
t0 = time.perf_counter()
nb_bc = GaussianNB()
nb_bc.fit(Xb_train, yb_train)
waktu_train_nb = time.perf_counter() - t0

t0 = time.perf_counter()
pred_nb = nb_bc.predict(Xb_test)
waktu_pred_nb = time.perf_counter() - t0

print(f"KNN        : akurasi={accuracy_score(yb_test, pred_knn):.4f}, "
      f"train={waktu_train_knn*1000:.2f} ms, prediksi={waktu_pred_knn*1000:.2f} ms")
print(f"GaussianNB : akurasi={accuracy_score(yb_test, pred_nb):.4f}, "
      f"train={waktu_train_nb*1000:.2f} ms, prediksi={waktu_pred_nb*1000:.2f} ms")
```

```
KNN        : akurasi=0.9561, train=1.18 ms, prediksi=45.28 ms
GaussianNB : akurasi=0.9386, train=3.32 ms, prediksi=1.43 ms
```

**Interpretasi Latihan 1:**

- **Akurasi:** KNN **0,9561** mengalahkan GaussianNB **0,9386** di dataset ini. Selisihnya tipis (0,0175), tetapi konsisten dengan sifat KNN yang fleksibel mengikuti pola lokal data.
- **Waktu pelatihan:** KNN jauh lebih cepat (sekitar 1 ms) karena ia lazy learner, training-nya hanya menyimpan data. GaussianNB butuh beberapa ms untuk menghitung mean dan varians tiap fitur per kelas.
- **Waktu prediksi:** kebalikannya. KNN butuh jauh lebih lama (45,28 ms) untuk 114 data uji karena setiap prediksi menghitung jarak ke 455 data latih. GaussianNB kilat (1,43 ms), tinggal menghitung probabilitas dari parameter yang sudah disimpan.

- **Kesimpulan:** tidak ada makan siang gratis. KNN menang akurasi di sini tetapi bayar mahal di waktu prediksi; GaussianNB sedikit kalah akurasi tetapi jauh lebih cepat dan ringan. Untuk aplikasi real-time dengan data besar, perbedaan ini menentukan.

## Latihan 2 Klasifikasi Teks dengan Multinomial Naive Bayes

**Soal:** Terapkan Multinomial Naive Bayes pada data teks sederhana yang sedikit lebih besar dari contoh 6.7. Di sini dipakai 16 kalimat Bahasa Indonesia (8 spam, 8 bukan spam), dibagi 75 persen latih dan 25 persen uji dengan train_test_split (stratify supaya proporsi spam seimbang, random_state=42). Ukur akurasinya, lalu uji dua kalimat baru.

```
teks2 = [
    "dapatkan diskon gratis sekarang", "menangkan hadiah gratis klik disini",
    "diskon spesial hari ini", "promo cashback besar-besaran",
    "klaim voucher belanja gratis", "bonus pulsa gratis untuk anda",
    "daftar segera dapatkan hadiah menarik", "obral besar potongan harga spesial",
    "rapat tim jam 10 pagi", "laporan keuangan bulan ini",
    "jadwal kuliah minggu depan", "tolong kirim berkasnya hari ini",
    "jadwal ujian sudah diumumkan", "jangan lupa bawa ktp besok",
    "mohon konfirmasi kehadiran rapat", "nilai tugas sudah diumumkan",
]
label2 = [1]*8 + [0]*8  # 1 = spam, 0 = bukan spam

X2_train, X2_test, y2_train, y2_test = train_test_split(
    teks2, label2, test_size=0.25, random_state=42, stratify=label2)

vectorizer2 = CountVectorizer()
X2_train_vec = vectorizer2.fit_transform(X2_train)
X2_test_vec = vectorizer2.transform(X2_test)

mnb = MultinomialNB(alpha=1.0)
mnb.fit(X2_train_vec, y2_train)
acc_teks = accuracy_score(y2_test, mnb.predict(X2_test_vec))
print(f"Jumlah data latih: {len(X2_train)}, data uji: {len(X2_test)}")
print(f"Akurasi pada data uji: {acc_teks:.4f}")

for kalimat in ["gratis ongkir hari ini", "rapat koordinasi besok pagi"]:
    hasil = mnb.predict(vectorizer2.transform([kalimat]))[0]
    print(f"'{kalimat}' -> {'Spam' if hasil==1 else 'Bukan Spam'}")
```

```
Jumlah data latih: 12, data uji: 4
Akurasi pada data uji: 0.7500
'gratis ongkir hari ini' -> Spam
'rapat koordinasi besok pagi' -> Bukan Spam
```

**Interpretasi Latihan 2:**

- Dari 16 kalimat, 12 dipakai latih dan 4 dipakai uji. Akurasi pada data uji adalah **0,7500**, artinya 3 dari 4 kalimat uji diklasifikasikan dengan benar.

- Kalimat "gratis ongkir hari ini" diprediksi **Spam** dengan benar karena kata "gratis" sangat identik dengan spam pada data latih, sedangkan "rapat koordinasi besok pagi" diprediksi **Bukan Spam** dengan benar karena kosakatanya mirip kalimat non-spam.
- Akurasi 0,7500 terlihat sedang, tetapi wajar untuk data latih yang sangat kecil (12 kalimat). Dengan data latih yang lebih besar dan beragam, akurasi MultinomialNB pada klasifikasi teks biasanya jauh lebih tinggi.
- **Kesimpulan:** MultinomialNB praktis untuk klasifikasi teks Bahasa Indonesia sekalipun dengan data kecil, selama kosakata spam dan bukan spam cukup berbeda.

Selesai. Selamat belajar!